

From vibe coding to spec-driven development

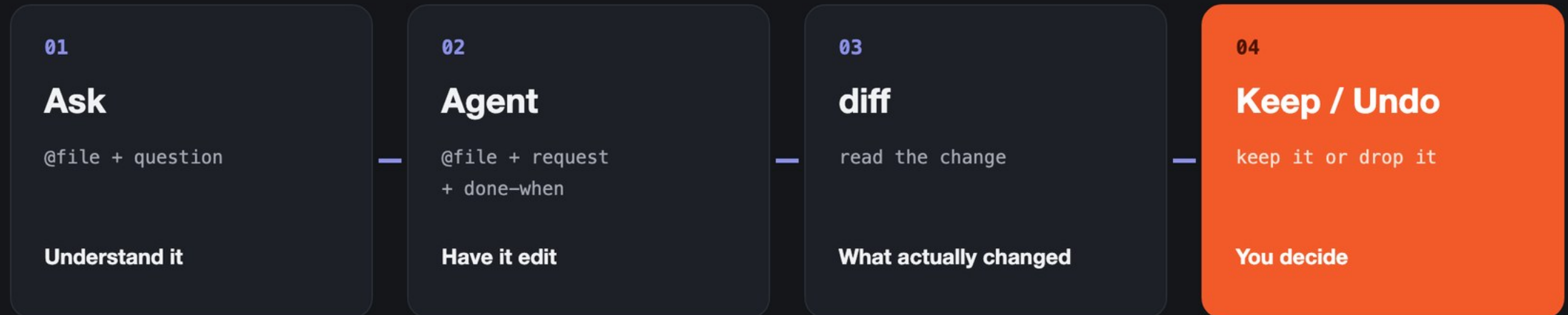
Build a memory game by asking for something “nice”.

Build poker by writing the requirements first.



LAST TIME

The loop hasn't changed



What changes is **what you hand it**. The loop itself is the same today.

WHAT WE DO TODAY

We build two games

The first is a simple game, with nothing decided up front. The second is a game with **far more to decide**, built after writing the requirements.

GAME 1

Memory game

Memory Match

**Nothing decided —
just “make it nice”**

VIBE CODING • Chapter 2

SHOT PENDING

Memory game, done
(sample)

GAME 2

Five-Card Draw poker

Five-Card Draw

**Requirements first,
then build**

SPEC-DRIVEN • Chapters 5 & 6

SHOT PENDING

Finished poker game
(sample)

What you take home isn't **which way is better** — it's **when to use which**.

TODAY'S PROMISES

Three things before we start

01

You don't have to finish

Stopping halfway through the second game is fine. Finished early? Take the stretch

02

The first one can be rough

We're asking for it sloppily on purpose

03

Stuck? Put your hand up

If someone's next to you, looking at their screen is faster

HOW THE 90 MINUTES RUN

Timetable

Time	Chapter	What we do	Who
0:00	Ch. 1 Today's goal	Announce that we build two games (5 min)	You
0:05	Ch. 2 Build it by vibe coding	The memory game from the three example lines (12 min)	All
0:17	Ch. 3 Compare	Compare the samples with your own (5 min)	All
0:22	Ch. 4 A late request	Put an extra request into the memory game (10 min)	All
0:32	(Wrap-up) What we've seen	Close out vibe coding → move to spec-driven (3 min)	You
0:35	Ch. 5 Write the requirements	Poker requirements via the requirements skill (15 min)	All
0:50	Ch. 6 Split into tasks, build two	Break it down → implement two by hand (15 min)	All
1:05	Ch. 7 Set a rule and let it work	Write a rule → carry on without the two lines (15 min)	All
1:20	Ch. 8 What you take home	When to use which • next session (10 min)	You

Hands on the keyboard for **72 of the 90 minutes**. Chapters 3 and 4 are **already down to 5 and 10 minutes**, so they don't get cut again.

CURSOR PRACTICAL COURSE • SESSION 2 OF 4

CHAPTER 01 • 0:00 5 min

Today's goal

We build two games. The first is simple, built by vibe coding. The second is hard, built from requirements.

TODAY'S GOAL

Four rungs

01 Get something running by vibe coding

Chapter 2

02 Notice that one request gave everyone something different

Chapter 3

03 Write requirements first for a game with a lot to decide

Chapters 5 & 6

04 Take a late request, add it to the requirements, ship it

STRETCH

Chapter 6

Reaching 01–03 is a good day. 04 is the stretch — missing it isn't failing.

TERMS

Two words, used consistently

Word	What it points at
Vibe coding	The name of a way of working : asking without deciding anything first. Today's first game (the memory game) is built this way
Spec-driven development	The name of a way of working : writing the requirements before you ask. Today's second game (poker) is built this way

Both are names for ways of working. The things you build keep their own names (“the memory game”, “poker”).

CHAPTER 1 COLUMN

We'll run Agent a lot today

Thirty seconds on usage before we start.

Even on Auto	you're billed at the price of whichever model it routed to
Every plan	comes with some usage included
Once that's gone	you either pay as you go, or move up a plan

Auto's settings (Cost / Balance / Intelligence) **change which model it routes to** — they aren't a discount. No prices in these slides. When it matters, check the official pricing page.

CURSOR PRACTICAL COURSE • SESSION 2 OF 4

CHAPTER 02 • 0:05 12 min

Build it by vibe coding

How far does it get when you decide nothing? Everyone ends up with their own memory game.

CHAPTER 2 YOUR TURN

Have it build a memory game (12 min)

- 1 Make a new folder called `session02/` (1 min)
- 2 Open a new chat and send the three lines below (10 min)
- 3 When it's done, open it in the browser (1 min)
Right-click in the sidebar → **Open In Browser**

```
Build me a memory match game.  
HTML + JS, playable in the browser.  
Make it nice.
```

“Make it nice” is exactly the **phrasing we said to avoid** in session 1. **We're being sloppy on purpose.** Leave the model on **Auto**.

SHOT PENDING

The memory-game request,
before sending

CHAPTER 2 SAMPLE

Minutes later, something runs

SAMPLE • yours will differ

SHOT PENDING

The finished memory game
(built by a learner)

Fast. Something working, right away. **This much is vibe coding at its best.**

CHAPTER 2 IF YOU'RE STUCK

Not running? Have Agent look

Cursor's browser is **something Agent can drive**. Opening pages, clicking, even **reading errors out of the console**.

```
Open the game I just built in the browser,  
and if there are console errors, tell me the cause.
```

Opening it in your own browser may not work. Open it in Cursor's built-in browser.

Browser actions need approval (by default it asks each time). In class, leave it as it is.

CURSOR PRACTICAL COURSE • SESSION 2 OF 4

CHAPTER 03 • 0:17 5 min

Compare

The same three lines produced different things. What you compare against is the samples on the slide, so it works even with nobody next to you.

CHAPTER 3 ★

Same three lines, different results

Run 1

SHOT PENDING

The memory game (run 1)

Run 2

SHOT PENDING

A different one from the same three lines (run 2)

All I did was send the same three lines to the same model twice. Still two different games. Yours will differ from both of these.

CHAPTER 3 YOUR TURN

Compare the two samples with yours

Compare yours against the two samples on the previous page. **The instructor leaves the samples up.**

What to compare	Sample 1	Sample 2	Yours
How many cards			
What's on the face (numbers / emoji / colours)			
Is there a score or a move count			
Is there a difficulty setting			
Look and feel (colours • layout)			

If someone's next to you, fill the same table in for theirs. If not, the samples are enough.

CHAPTER 3 YOUR TURN

Count what you never asked for (1 min)

You sent **three lines**, nothing more. List everything else that turned up.

What you asked for

That it's a memory game
That it runs on HTML + JS
That it looks nice

What came along uninvited

The card count
The card faces
Score • move count
A difficulty setting
Colours • layout

Nobody asked for a score, or difficulty levels. **You can't tell what you specified from what the AI decided.**

CURSOR PRACTICAL COURSE · SESSION 2 OF 4

CHAPTER 04 · 0:22 10 min

A late request

The peak of the session. A new request arrives from the client — what happens to the game you just built?

CHAPTER 4 YOUR TURN

Open a new chat (1 min)

- 1 Not the chat you're in — open a new one**
The current chat still holds the context of what you just built
- 2 A new chat doesn't have that context**
It's the same situation as handing work to someone else

Skip this and the chapter doesn't work. The whole point is to create a “handing it to someone with no context” situation.

SHOT PENDING

Where to open a new chat

CHAPTER 4 FROM THE CLIENT

Two more requests came in

- ① When a pair matches, let the player flip again.
- ② From the third mistake onward, shorten the wait before the cards flip back to 0.5 seconds.

Neither of these is a **rule the AI would infer from what memory games usually do**. So not one of them is in what you just built.

CHAPTER 4 YOUR TURN

Have it implement the request (6 min)

Send the two items from the previous page as they are.

```
@session02/
```

- ① When a pair matches, let the player flip again.
 - ② From the third mistake onward, the wait before the cards flip back is 0.5 seconds.
- Don't change any other behaviour.

Send it in the new chat. Not as a continuation of the old one.

CHAPTER 4 YOUR TURN

Write it down (3 min)

What to record	Answer
Lines changed (+/- in the diff)	
Did anything existing break	
Was it implemented correctly — and could you judge that yourself	

The third row is the most important question of the day.

CHAPTER 4

How did you check that the third mistake onward really is 0.5 seconds?

Running it tells you nothing. Your eyes can't tell 0.5 seconds from 1 second.

To be sure, **you have to open the code and hunt for the number in the timer.**

CHAPTER 4

You opened a new chat a minute ago.

The AI in that chat **had no idea** what you were trying to build.

It read the code, **guessed**, and edited.

Whether eight cards was your decision or the AI's

Whether the one-second flip-back was intentional or an accident

None of it is written down, so whoever reads it can't know.

This time the one you handed it to happened to be an AI. Tomorrow's you, or the person next to you, is in exactly the same position.

CHAPTERS 2-4 WRAP-UP

What you handed over, what came back

Let's count what actually happened in chapter 2.

	Contents
What you handed over	Three lines (the game's name / HTML + JS / “make it nice”)
What you decided	That it's a memory game. That it runs in a browser. That's all
What came back	One playable game (in minutes)
What the AI decided	Card count • face design • layout • colours • score or no score • difficulty • how long until they flip back.....

The AI decided far more than you did.

CHAPTERS 2-4 WRAP-UP

In one sentence

**Vibe coding is fast.
But you can't control it.**

The “fast” from chapter 2 is a real strength. The other three all fold into this one line.

Every run produced **something different**



Can't aim

You **can't tell** what the AI decided



Can't hold it

You **can't judge** whether it went in



Can't verify

Aim, hold, verify. Control is all three at once.

CHAPTERS 2-4 WRAP-UP

Why you can't control it

Because you never decided.

What was never decided	So
Neither the card count nor the faces were decided	Can't aim
There's no record of what you asked for	Can't hold it
“What counts as done” was never decided	Can't verify

All anyone said in chapter 4 was **“make it 0.5 seconds”**. **What counts as done** was never decided by anyone. So running it tells you nothing — **your eyes can't tell 0.5 seconds from 1 second. The screen won't hand you the answer.**

CHAPTERS 2-4 WRAP-UP

So: write it first

To take control back, we decide first.

	Vibe coding	Requirements first
Aim	What you decided is in your head	What you decided is written down
Verify	There's no “what counts as done”	Done-when, written so the screen can show it
Hand over	Only the person who built it can carry on	Anyone can pick it up

Chapter 5 Write the requirements (this time for a game with far more to decide)
Chapter 6 Build from the requirements
Chapter 7 Set the project rule

Vibe coding isn't the villain. The memory game came out in minutes — a real strength. What changes is **the scale, and the amount to decide.**

CURSOR PRACTICAL COURSE • SESSION 2 OF 4

CHAPTER 05 • 0:35 15 min

Write the requirements

The second game. Something much harder than the memory game, built from requirements.

CHAPTER 5 EXPLANATION

A Skill is “the procedure for this kind of work”

So you stop writing out the same steps every time. Where it lives is fixed.

Kind	Path	Scope
Project	.cursor/skills/<name>/SKILL.md	This repo only
Personal	~/.cursor/skills/<name>/SKILL.md	All of your projects

How to call it	Scope	Action
Automatic	When Agent decides it should	Nothing (the description decides)
Slash	That one message only	Type / in the input → pick the Skill
Custom Mode	The whole session	Alt+Enter (Option+Enter on Mac)

Both Skills we use today are already in the repo (git pull). Today we call them with a slash.

CHAPTER 5 EXPLANATION

Inside, it's just Markdown

With two lines of settings on top.

```
---
name: requirements
description: use when you need to draw out what someone
  wants built and fold it into four requirement sections
---

# Writing requirements

(everything below is an ordinary set of steps)
```

Field	Role
name	Lowercase, digits and hyphens only. Must match the parent folder name
description	The important one. Agent reads this to decide whether to use it in a conversation

Those two are the only required fields.

CHAPTER 5 EXPLANATION

The two Skills we use today

Skill	What it does	What it doesn't
requirements	Draws out what you want built and folds it into what to build / screen / interaction / what we won't build	Writes no code
task-breakdown	Splits the requirements into 3–5 tasks with done-when conditions and saves <code>tasks.md</code>	Writes no code

Neither implements anything. They're tools for deciding.

CHAPTER 5 INSTRUCTOR DEMO ★

Asking for poker with “make it nice”

Three lines, that's the whole request. **Exactly how we asked** for the memory game in chapter 2.

```
Build me a poker game.  
HTML + JS, playable in the browser.  
Make it nice.
```

SAMPLE · yours will differ

SHOT PENDING

Poker built from “make it nice”
(CREDIT / BET / payable · no opponent)

CHAPTER 5 INSTRUCTOR DEMO

It runs. But what is it?

What came along uninvited

Chips (CREDIT 1000)

A bet selector

A nine-tier paytable

What you thought you asked for, and isn't there

An opponent

Is this the poker game you wanted?

Where is the opponent?

Who asked for chips and betting?

CHAPTER 5 EXPLANATION

Four sections every requirement needs

Section	Contents
What to build	What it takes for the thing to hold together
Screen	What you can see, what you can press
Interaction	What happens when the user does something
What we won't build	What you decide is out of scope this time

The fourth is where today's leverage is. **The AI adds things you never wrote down**, so rule them out in advance.

CHAPTER 5 EXPLANATION

What splits in poker if you don't decide

Unlike a memory game, poker has **a lot to decide**.

Splits if undecided	For example
The opponent	Is there one at all? If so, how many CPUs, and how do they draw
Hand rankings	How far down the ten tiers you implement
Ties between the same hand	One pair against one pair — who wins
The draw	How many cards / how many times / can you discard none
Betting	Are chips in play at all

Leave it alone and the AI decides all of it. Whether what it decided matches what you meant is something you can't check without requirements.

CHAPTER 5 HANDOUT

Hand rankings (strongest first)

The outlined cards are the ones making the hand. Hand names stay in **English** throughout.

1 **Royal Flush**
the top hand
10 • J • Q • K • A of one suit



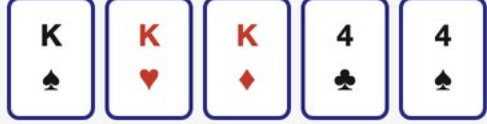
2 **Straight Flush**
straight + flush
Five in a row, all one suit



3 **Four of a Kind**
quads
Four cards of the same rank



4 **Full House**
a boat
Three of a rank + a pair



5 **Flush**
all one suit
Five cards of the same suit



6 **Straight**
a run of five
Five ranks in a row



7 **Three of a Kind**
trips
Three cards of the same rank



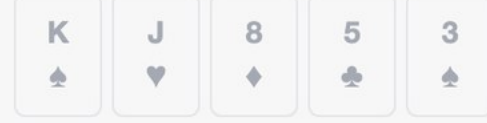
8 **Two Pair**
two pairs
Two pairs of matching ranks



9 **One Pair**
a pair
Two cards of the same rank



10 **High Card**
no hand
None of the above



How far down this table you build is yours to decide in the requirements. You don't have to do all of it.

CHAPTER 5 YOUR TURN

Call the requirements skill and fill it in

① In a new chat, type `/` in the input and pick **requirements** (2 min).

```
/requirements I want to build five-card draw poker
```

If it isn't in the list, you haven't run `git pull`.

② Answer what the skill asks (8 min). Four sections to fill.

- What to build — what it takes for the game to hold together
- Screen — what you can see, what you can press
- Interaction — what happens when the user does something
- What we won't build — what you decide is out of scope

SHOT PENDING

The skill list open on `/`

CHAPTER 5 THE LEVERAGE

Things to put under “what we won't build”

Nothing coming to mind? Pick from here.

Betting • chips • stakes

Bluffing • CPU strategy

Three or more players

Jokers

Animation (card-flip effects)

Saving results

The AI adds things you never wrote down. Rule them out first and it won't.

CHAPTER 5 FROM THE CLIENT

One more request

When a hand is decided, outline the cards that make up that hand.
Grey out the cards that aren't part of it.

Handle it by **adding one line to the requirements**. Don't touch the code.

CURSOR PRACTICAL COURSE • SESSION 2 OF 4

CHAPTER 06 • 0:50 15 min

Split into tasks, build two

Cut the requirements into pieces small enough to hand over one at a time. Just two here. The rest comes in chapter 7, once the rule is working.

CHAPTER 6 EXPLANATION

One thing per request

Session 1 covered “a vague request gives you a vague result”. **Breaking work into tasks is how you actually do that.**

```
[Goal] one line
[Target] @file or folder
[Constraints] what must not break
[Done when] what has to be true to stop
```

With a done-when condition	you can judge what comes back
When it gets long	switch to a new chat

This is the counterpart to chapter 4, where the standard for judging lived only in your head.

CHAPTER 6 EXPLANATION

What to look at before Keep

The loop is the same as session 1. Exactly one thing is added.

	Session 1	Today
After you ask	read the diff	read the diff
Before Keep	did only what you meant change	+ is the done-when condition met
The standard	your own eyes	the done-when condition, written down

In chapter 4 you could only check it against your own memory because **there was nothing to check against**. Today there is.

When it builds something new, the diff is all additions. There you only look at “does it run” and “are there too many files”. **When it adds to what exists** (chapter 4, task 2 onward) you read it like session 1.

CHAPTER 6 YOUR TURN

Make a folder, split it into tasks

1 Make a working folder (1 min)

`session02-spec/`. Don't mix it with the memory game

2 Split it into tasks (4 min)

Hand the requirements you wrote to `/task-breakdown`

SHOT PENDING

The result of `/task-breakdown`
(split into 3–5 tasks)

Three to five tasks is plenty. Too many? Cut some. More than ten means the requirements are too big.

CHAPTER 6 YOUR TURN

Ten minutes, two turns of this loop

The split is saved in `session02-spec/tasks.md`. **Two turns only here. The rest continues in chapter 7.**

- 1 Open tasks.md and pick one task that isn't done
- 2 Open a new chat
- 3 Paste that task in and send it
- 4 When it comes back, read the diff
- 5 Check whether the done-when condition is met
- 6 If both are fine, Keep
- 7 Mark that task in tasks.md
- 8 Back to 1

Don't skip step 2, the new chat. With the previous task's context still there, it starts editing things you never asked about. **One task, one chat.**

CHAPTER 6 YOUR TURN

This is what you send each time

It's what step 3 above sends. Copy it out of `tasks.md`.

```
(copy the task's "what to build" from tasks.md)
Done when: (copy that task's done-when condition from tasks.md)
Don't change any other feature.
```

When it comes back: **read the diff** → **check the done-when condition** → **Keep**.
Then the next task.

SHOT PENDING

One task handed over
(with its done-when condition)

CHAPTER 6 YOUR TURN

Read the diff, then Keep

SHOT PENDING

Reading the diff and pressing Keep

The same as session 1. Except today, **after reading the diff you check it against the done-when condition**. Both fine, then Keep.

CURSOR PRACTICAL COURSE • SESSION 2 OF 4

CHAPTER 07 • 1:05 15 min

Set a rule and let it work

The two lines you typed every time in chapter 6 now sit in one place. Then you finish the remaining tasks without typing them.

CHAPTER 7 EXPLANATION

The third mechanism we touch today

Mechanism	When it applies	Where it lives	Today that's
Rules	Always	.cursor/rules/*.mdc	what's next
Skills	Only when called	.cursor/skills/...	requirements / task-breakdown
Requirements	That project only	an ordinary file	the poker requirements

Skills and requirements we've touched today. Only Rules is left.

CHAPTER 7 EXPLANATION

What you typed every time can just sit there

In chapter 6 you **typed these two lines every single time** you handed over a task. Once per task, the same thing.

```
Done when: (copy that task's done-when condition from tasks.md)  
Don't change any other feature.
```

`.cursor/rules/` is where you put things so they're not in every prompt.

CHAPTER 7 EXPLANATION

How to write a rule

Put an `.mdc` file in `.cursor/rules/`. It's Markdown with settings on top, nothing more.

```
---
description: the short line shown in the rules list
globs: only apply when touching files matching this pattern
alwaysApply: true means it's always loaded
---

# Heading

- what to do
- what not to do
```

Tips	Contents
Short and concrete	5–15 lines you can actually follow
Say do and don't	vague principles don't get followed
Don't overuse alwaysApply	always loaded means always eating context

CHAPTER 7 YOUR TURN

Write one rule (3 min)

Send Agent the following as it is.

```
Create .cursor/rules/task-cycle.mdc.  
Set alwaysApply: true.  
The contents are exactly these four lines. Don't add anything else.  
  
- work on only one task at a time  
- after implementing, check it against that task's done-when condition  
- once the done-when condition is met, stop. Don't move on to the next task  
- don't add features that weren't asked for
```

Look at the fourth line. “Don't add features that weren't asked for” — **the thing that's been biting us all day.**

CHAPTER 7 YOUR TURN

Carry on without the two lines (9 min)

Picking up chapter 6. Implement the remaining tasks. **Except this time, this is all you send.**

(copy only the task's "what to build" from tasks.md)

	Chapter 6	Now
Typed every time	what to build + done-when + don't change anything else	what to build only
The done-when condition	written in the prompt	in tasks.md (the AI reads it)
"Don't change anything else"	written in the prompt	in the rule

The steps are the same as chapter 6. Pick one from tasks.md → new chat → send → read the diff → check the done-when condition → Keep → mark it off.

CHAPTER 7

The finish line

- ☐ Five cards are dealt and visible on screen
- ☐ You can pick cards and draw
- ☐ The hand name is displayed
- ☐ Chapter 5's extra request (outlining the cards making the hand) is in

Getting to the third one is plenty. The fourth is the stretch (rung 04 of today's goal).

Stop halfway and someone else can carry on. The requirements, the tasks and the **rule** are all written down. With the memory game, **only you can carry on.** That's the difference.

CHAPTER 7 SAMPLE ★

The outline tells you whether it's right

SAMPLE • yours will differ

SHOT PENDING

The finished poker game
(cards making the hand are outlined)

In chapter 4, checking meant **going back to your own memory**. Now **the screen tells you**. You moved the standard for judging out of your head and onto the screen.

CHAPTER 7 COLUMN

For anyone who finished early (Design Mode)

Want to fix the look? There's a way to **point** at it instead of describing it. Press it with your game open in the built-in browser.

Action	Keys
Toggle Design Mode	Ctrl+Shift+D (Cmd+Shift+D on Mac)
Select an area	Shift + drag
Add the selection to the chat	Ctrl+L (Cmd+L on Mac)

Good for **look, spacing and layout**. Not for **calculation logic like scoring hands** (just ask Agent for that).

CURSOR PRACTICAL COURSE • SESSION 2 OF 4

CHAPTER 08 • 1:20 10 min

What you take home

When to use which, and what's next. This chapter never gets shortened.

CHAPTER 8

Hand-in (30 seconds)

One screenshot of your
poker game, into the chat
or the shared folder.

Outlined cards are enough.
It doesn't have to get as far as deciding a winner.

SHOT PENDING

A sample hand-in

CHAPTER 8

What to take home today

01

Vibe coding is fast. But you can't control it

can't aim / can't hold it / can't verify

02

Requirements go in a form you can hand to someone else

what to build + what we won't build

03

Leave it in a form you can judge

move the standard out of your head, onto the screen or into a done-when condition

CHAPTER 8

Picking between them

When it's like this	Use
Small • throwaway • only you will see it	Vibe coding (about the size of the memory game)
A lot to decide • built with others • edited later	Spec-driven development (poker-sized and up)

The line isn't “size” — it's “how much there is to decide”. The more there is to decide, the more control you're handing away.

NEXT TIME • SESSION 3

Building as a team

You pick your own theme and start building an app.

What happens when today's “write the requirements first” meets a team.

Plus a minimal branch • PR • review flow.

CHAPTER 8 BEFORE NEXT TIME

Two things to check right now

The first one you can check on the Cursor screen alone. You've touched files today, so **a number on the Source Control icon means you're fine.**

- ☐ “Source Control” in the sidebar shows today's changes
- ☐ You have a GitHub account

If either is missing, **put your hand up.** It decides how session 3 is run.
